

Materia: Microservicios

Docente: MSc. Viviana Flores

Duración: 2 horas

Modalidad: Parejas (máximo 2 integrantes)

1. INSTRUCCIONES GENERALES

- El trabajo se realizará en parejas. No se aceptan grupos de más de dos integrantes.
- La evaluación tiene una duración máxima de 2 horas.
- Cada pareja debe desarrollar el sistema desde cero.
- Queda prohibido el uso de código de terceros ajeno a los frameworks indicados (Spring Boot, Nginx, PostgreSQL, Docker).
- La comunicación entre compañeros de otras parejas está prohibida durante la evaluación.

Entregables obligatorios

- Enlace al repositorio de GitHub (público) con el código fuente completo.
- Video de demostración funcional (ver requisitos a continuación).

Requisitos del video de demostración

- Duración máxima: 10 minutos.
- Debe mostrar todos los contenedores activos en Docker Desktop.
- Debe ejecutar en vivo cada endpoint requerido desde Postman a través del API Gateway.
- Debe evidenciar la comunicación entre microservicios (crear un pedido con un plato válido y uno inválido).
- El video puede subirse a YouTube (no listado) o Google Drive con acceso habilitado.

Penalizaciones

- Video sin demostración de la comunicación entre servicios.
- Repositorio privado o enlace inaccesible: el criterio correspondiente se califica como Insuficiente.

2. CONTEXTO DEL SISTEMA — RestauranteTech

RestauranteTech es una empresa de tecnología que necesita digitalizar la gestión de pedidos de una cadena de restaurantes. La solución debe construirse bajo una arquitectura de microservicios para garantizar escalabilidad, desacoplamiento y mantenibilidad independiente de cada componente.

Como equipo de desarrollo, usted(es) deberá(n) construir dos microservicios independientes que se comuniquen entre sí, orquestados por un API Gateway basado en Nginx y desplegados mediante Docker Compose.

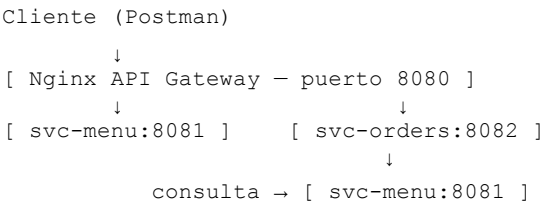
Objetivo del sistema: Permitir la gestión del menú de platos disponibles y la creación de pedidos, garantizando que solo se puedan ordenar platos existentes y activos en el menú.

3. ARQUITECTURA REQUERIDA

El sistema debe implementar la siguiente arquitectura de microservicios:

Componente	Tecnología	Puerto
API Gateway	Nginx	8080
svc-menu	Spring Boot + PostgreSQL	8081
svc-orders	Spring Boot + PostgreSQL	8082
svc-inventory (PUNTO EXTRA)	Spring Boot + PostgreSQL	8083

Diagrama de flujo de comunicación



Regla clave: Todo el tráfico externo entra únicamente por el API Gateway en el puerto 8080. Los microservicios NO deben exponerse directamente al cliente; se comunican entre sí dentro de la red Docker.

4. ESPECIFICACIÓN DE MICROSERVICIOS

4.1 svc-menu — Servicio de Menú

Puerto: 8081 | **Base de datos:** db_menu | **Puerto BD:** 5433

Responsable de gestionar el catálogo de platos del restaurante. Expone operaciones CRUD completas sobre la entidad Dish.

Modelo: Dish

Campo	Tipo	Restricciones	Descripción
id	Long	PK, auto-generado	Identificador único del plato
name	String	NOT NULL, max 100 chars	Nombre del plato
description	String	Opcional, max 255 chars	Descripción del plato
category	String	NOT NULL	Categoría: ENTRADA, PLATO_FUERTE, POSTRE, BEBIDA
price	Double	NOT NULL, > 0	Precio del plato en USD
available	Boolean	NOT NULL, default true	Indica si el plato está disponible

Endpoints requeridos — svc-menu

Método	Ruta (vía Gateway)	Body / Params	Código HTTP	Respuesta
POST	/api/menu/dishes	JSON: Dish (sin id)	201	Dish creado con id
GET	/api/menu/dishes	—	200	Lista de todos los Dish
GET	/api/menu/dishes/{id}	Path: id	200/404	Dish o mensaje de error
PUT	/api/menu/dishes/{id}	JSON: Dish actualizado	200	Dish actualizado
DELETE	/api/menu/dishes/{id}	Path: id	204	Sin cuerpo

4.2 svc-orders — Servicio de Pedidos

Puerto: 8082 | **Base de datos:** db_orders | **Puerto BD:** 5434

Responsable de registrar y consultar pedidos. Antes de guardar un pedido, debe consultar svc-menu para validar la existencia y disponibilidad del plato, y obtener su precio para calcular el total.

Modelo: Order

Campo	Tipo	Restricciones	Descripción
id	Long	PK, auto-generado	Identificador único del pedido
customerName	String	NOT NULL, max 100 chars	Nombre del cliente o mesa
dishId	Long	NOT NULL	ID del plato solicitado (referencia a svc-menu)
quantity	Integer	NOT NULL, >= 1	Cantidad de platos solicitados

total	Double	Calculado automáticamente	price × quantity (obtenido de svc-menu)
status	String	NOT NULL, default PENDING	Estado: PENDING, CONFIRMED, CANCELLED
createdAt	LocalDateTime	Auto-generado	Fecha y hora de creación del pedido

Endpoints requeridos — svc-orders

Método	Ruta (vía Gateway)	Body / Params	Código HTTP	Respuesta
POST	/api/orders	JSON: customerName, dishId, quantity	201	Order creado con total calculado
GET	/api/orders	—	200	Lista de todos los pedidos
GET	/api/orders/{id}	Path: id	200/404	Order o mensaje de error

5. COMUNICACIÓN ENTRE MICROSERVICIOS

La comunicación entre svc-orders y svc-menu es sincrónica mediante llamadas HTTP REST. svc-orders actúa como consumidor de la API de svc-menu usando WebClient de Spring Boot.

Flujo completo: creación de un pedido

Paso	Actor	Acción
1	Cliente → Nginx (puerto 8080)	POST /api/orders con { customerName, dishId, quantity }
2	Nginx → svc-orders (puerto 8082)	Nginx redirige la solicitud internamente
3	svc-orders → svc-menu (puerto 8081)	GET http://svc-menu:8081/api/menu/dishes/{dishId}
4a	svc-menu → svc-orders (respuesta OK)	Retorna el objeto Dish con precio y available=true
4b	svc-menu → svc-orders (error)	Retorna 404 si el plato no existe o available=false
5a	svc-orders guarda el pedido	Calcula total = price × quantity, persiste en db_orders, retorna 201
5b	svc-orders retorna error	Retorna 400 o 404 con mensaje descriptivo al cliente

Reglas de validación en svc-orders

- Si el plato NO existe en svc-menu (respuesta 404): retornar error 404 con mensaje: "El plato con id {dishId} no existe en el menú."
- Si el plato existe pero available = false: retornar error 400 con mensaje: "El plato '{name}' no está disponible actualmente."
- Si el plato es válido y disponible: calcular total = price × quantity y guardar el pedido.

6. ESTRUCTURA DE CARPETAS ESPERADA

Su repositorio de GitHub debe contener la siguiente estructura de carpetas y archivos:

```

restaurantetech/
├── docker-compose.yml
├── nginx.conf
├── svc-menu/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/
│       │   │   ├── com/restaurantetech/menu/
│       │   │   │   ├── controller/
│       │   │   │   │   └── DishController.java
│       │   │   │   ├── model/
│       │   │   │   │   └── Dish.java
│       │   │   │   ├── repository/
│       │   │   │   │   └── DishRepository.java
│       │   │   │   └── service/
│       │   │   │       └── DishService.java
│       │   └── resources/
│       │       └── application.properties
│   └── svc-orders/
│       ├── Dockerfile
│       ├── pom.xml
│       └── src/
│           ├── main/
│           │   ├── java/
│           │   │   ├── com/restaurantetech/orders/
│           │   │   │   ├── client/
│           │   │   │   │   └── MenuServiceClient.java
│           │   │   │   ├── controller/
│           │   │   │   │   └── OrderController.java
│           │   │   │   ├── model/
│           │   │   │   │   └── Order.java
│           │   │   │   ├── repository/
│           │   │   │   │   └── OrderRepository.java
│           │   │   │   └── service/
│           │   │   │       └── OrderService.java
│           │   └── resources/
│           │       └── application.properties
│   └── README.md

```

Nota: puede agregar archivos adicionales (DTOs, configuraciones), pero la estructura mínima anterior es obligatoria.

7. PUNTO EXTRA — svc-inventory (Servicio de Inventario)

Valor: +1 punto adicional sobre la nota final. Este componente es completamente opcional.

Implemente un tercer microservicio svc-inventory que gestione el stock de ingredientes/platos disponibles. svc-orders debe consultarlo antes de confirmar un pedido, verificando que haya stock suficiente, y decrementar el stock al confirmar la orden.

Modelo: InventoryItem

Campo	Tipo	Restricciones	Descripción
id	Long	PK, auto-generado	Identificador único
dishId	Long	NOT NULL	Referencia al plato en svc-menu
stockQuantity	Integer	NOT NULL, >= 0	Cantidad disponible en inventario
lastUpdated	LocalDateTime	Auto-generado	Última actualización del stock

Integración esperada

- svc-orders consulta svc-inventory: GET `http://svc-inventory:8083/api/inventory/{dishId}`

- Si stockQuantity < quantity solicitada: retornar error 400 con mensaje descriptivo.
- Si hay stock suficiente: decrementar el stock (PUT) y proceder a crear el pedido.
- Nginx debe incluir la ruta /api/inventory/ apuntando a svc-inventory:8083.

8. CRITERIOS DE EVALUACIÓN

#	Criterio	Descripción	Puntos
1	svc-menu funcional	Los endpoints CRUD de svc-menu responden correctamente a través del API Gateway.	2
2	svc-orders funcional	Los endpoints de svc-orders responden correctamente a través del API Gateway.	1
3	Comunicación entre servicios	svc-orders consulta correctamente a svc-menu; valida existencia, disponibilidad y calcula el total.	2
4	Manejo de errores	Se retornan mensajes descriptivos cuando el plato no existe o no está disponible.	1
5	Docker Compose y Nginx	Todos los contenedores levantan correctamente; el tráfico fluye a través del API Gateway.	2
6	Video de demostración	El video muestra el sistema funcionando, incluyendo escenarios de éxito y error.	2
TOTAL			10 puntos

Nota: el video de demostración tiene un valor importante en la calificación. Su ausencia o una demostración incompleta afecta directamente la calificación final.

RÚBRICA DE EVALUACIÓN

Criterio	Excelente (10 – 9)	Bueno (8.9 – 7.5)	Suficiente (7.4 – 6)	Insuficiente (< 6)
svc-menu funcional (2 pts)	Todos los endpoints CRUD responden correctamente via Gateway. El modelo Dish incluye todos los campos requeridos. Las categorías y validaciones funcionan.	Los endpoints principales (POST, GET) funcionan. Falta algún endpoint menor (PUT o DELETE) o alguna validación.	Al menos GET y POST funcionan con datos básicos. El modelo está incompleto o las rutas no pasan correctamente por Nginx.	svc-menu no levanta, no tiene endpoints funcionales o no es accesible desde el API Gateway.
svc-orders funcional (1 pt)	Los tres endpoints responden correctamente. El modelo Order incluye todos los campos. El status por defecto y createdAt se generan automáticamente.	POST y GET /api/orders funcionan. Falta GET por id o algún campo del modelo.	Solo POST funciona. El modelo Order está incompleto pero permite crear un pedido básico.	svc-orders no levanta o no tiene endpoints funcionales.
Comunicación entre servicios (2 pts)	svc-orders consulta correctamente a svc-menu. Valida existencia del plato, verifica available=true, obtiene el precio y calcula total = price × quantity correctamente.	svc-orders consulta a svc-menu y valida existencia. El cálculo del total es correcto pero no verifica el campo available.	svc-orders intenta llamar a svc-menu pero la comunicación es parcial o el total no se calcula usando el precio real del servicio.	No existe comunicación entre servicios. svc-orders no consume la API de svc-menu.
Manejo de errores (1 pt)	Se retorna 404 con mensaje descriptivo si el plato no existe. Se retorna 400 con mensaje descriptivo si el plato no está disponible. Los mensajes identifican claramente el problema.	Se maneja al menos uno de los dos casos de error con mensaje descriptivo. El otro caso retorna un error genérico.	Los errores se manejan con códigos HTTP correctos pero sin mensajes descriptivos, o viceversa.	No se maneja ningún caso de error. El sistema lanza excepciones no controladas o retorna 500.
Docker Compose y Nginx (2 pts)	docker-compose up levanta todos los contenedores sin errores. Las rutas /api/menu/ y /api/orders/ funcionan correctamente a través del puerto 8080. Las redes Docker están configuradas correctamente.	Los contenedores levantan pero alguno requiere reintento. El Gateway funciona para la mayoría de rutas pero alguna ruta falla.	Los contenedores levantan con intervención manual. El Gateway funciona parcialmente o los servicios se probaron directamente sin pasar por Nginx.	Los contenedores no levantan o el archivo docker-compose.yml no existe / tiene errores graves que impiden el despliegue.
Video de demostración (2 pts)	El video muestra ambos integrantes presentándose. Se evidencian todos los contenedores activos en Docker. Se prueban todos los endpoints en Postman via Gateway, incluyendo escenarios de éxito y error. Duración adecuada.	El video muestra la mayoría de endpoints. Falta algún escenario de error o algún endpoint menor. Los contenedores se muestran activos.	El video es incompleto: muestra solo algunos endpoints o no evidencia la comunicación entre servicios. No se muestran ambos integrantes.	El video no fue entregado, no es accesible (enlace inválido) o no muestra el sistema funcionando.

MSc. Viviana Flores
Docente

Mg. Wilmer Albarracín
Director de Carrera